



ML & DL

23010101014 | Vishal Baraiya |
635

Lab - 7

Do all pre processing on diabetes csv

```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn import tree
```

Import diabetes.csv data ser using sklearn

```
In [2]: df = pd.read_csv('diabetes.csv')
df
```

Out[2]:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFu
0	6	148	72	35	0	33.6	
1	1	85	66	29	0	26.6	
2	8	183	64	0	0	23.3	
3	1	89	66	23	94	28.1	
4	0	137	40	35	168	43.1	
...	...	...	...	...	...	...	
763	10	101	76	48	180	32.9	
764	2	122	70	27	0	36.8	
765	5	121	72	23	112	26.2	
766	1	126	60	0	0	30.1	
767	1	93	70	31	0	30.4	

768 rows × 9 columns



Check the distribution of the target

```
In [3]: df["Outcome"].value_counts()
```

```
Out[3]: Outcome
Non Diabetic    500
Diabetic        268
Name: count, dtype: int64
```

Replace zeros with NaN for specific columns

```
In [4]: columns_to_replace = ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI']
df[columns_to_replace] = df[columns_to_replace].replace(0, np.nan)
```

Check for missing values

```
In [5]: print(df.isnull().sum())
```

```
Pregnancies      0
Glucose           5
BloodPressure     35
SkinThickness    227
Insulin          374
BMI              11
DiabetesPedigreeFunction  0
Age              0
Outcome          0
dtype: int64
```

Fill missing values with median

```
In [6]: df.fillna(df.median(numeric_only=True), inplace=True)
df.head()
```

```
Out[6]:
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	6	148.0	72.0	35.0	125.0	33.6	0.351	31.0	0
1	1	85.0	66.0	29.0	125.0	26.6	0.681	33.0	0
2	8	183.0	64.0	29.0	125.0	23.3	0.351	31.0	0
3	1	89.0	66.0	23.0	94.0	28.1	0.178	33.0	0
4	0	137.0	40.0	35.0	168.0	43.1	0.351	31.0	0

```
In [7]: print(df.isnull().sum())
```

```
Pregnancies      0
Glucose           0
BloodPressure     0
SkinThickness     0
Insulin           0
BMI              0
DiabetesPedigreeFunction  0
Age              0
Outcome          0
dtype: int64
```

Convert Targer data into interger code

```
In [8]: df['Outcome'] = df['Outcome'].map({'Non Diabetic': 0, 'Diabetic': 1})
df.head()
```

Out[8]:

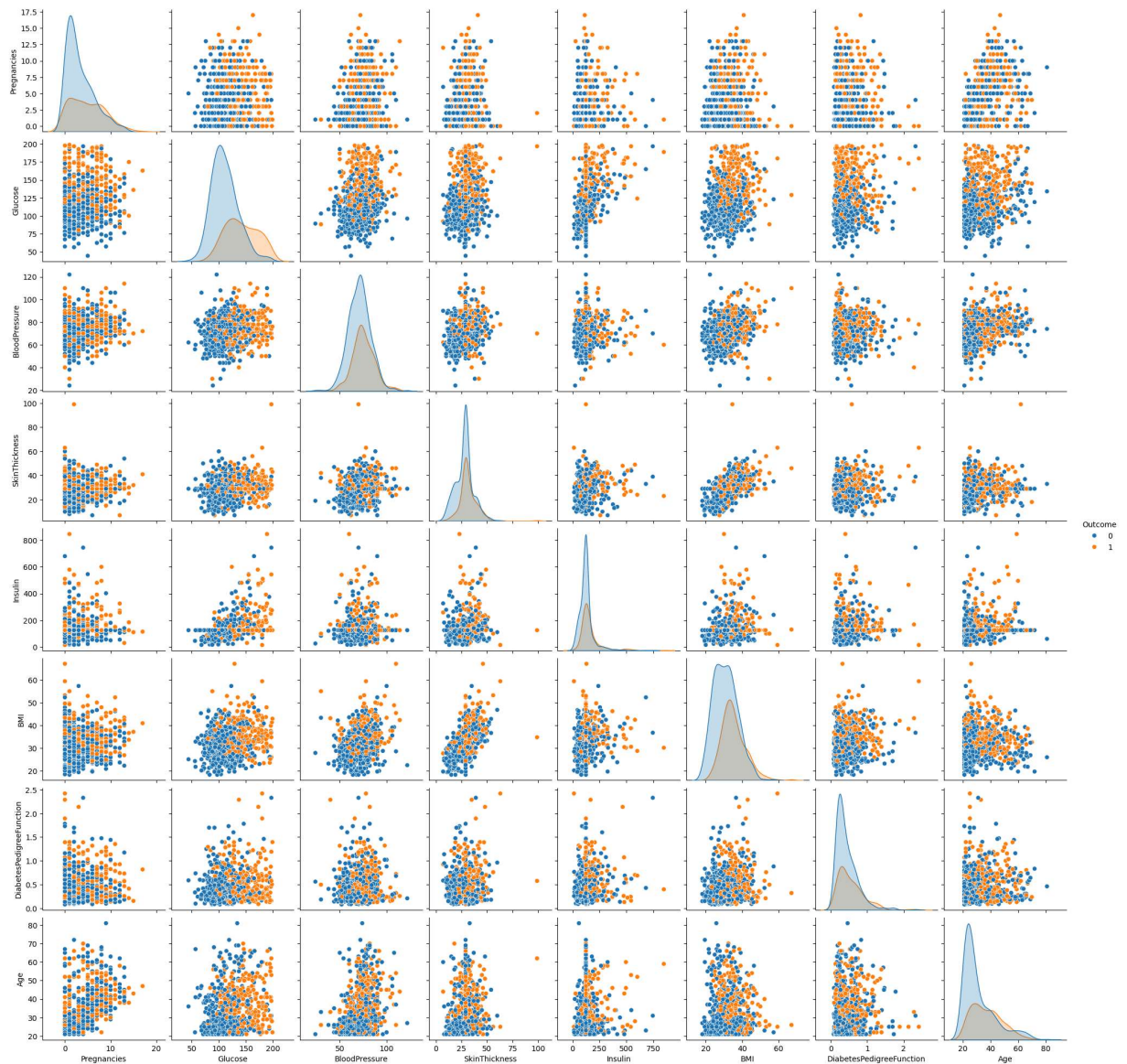
	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunc
0	6	148.0	72.0	35.0	125.0	33.6	C
1	1	85.0	66.0	29.0	125.0	26.6	C
2	8	183.0	64.0	29.0	125.0	23.3	C
3	1	89.0	66.0	23.0	94.0	28.1	C
4	0	137.0	40.0	35.0	168.0	43.1	2



Visualize Distributions

```
In [9]: sns.pairplot(
    df,
    hue="Outcome",          # 0 = Non Diabetic, 1 = Diabetic
    # diag_kind="kde",      # density curve on diagonal
    # palette="Set1"
)

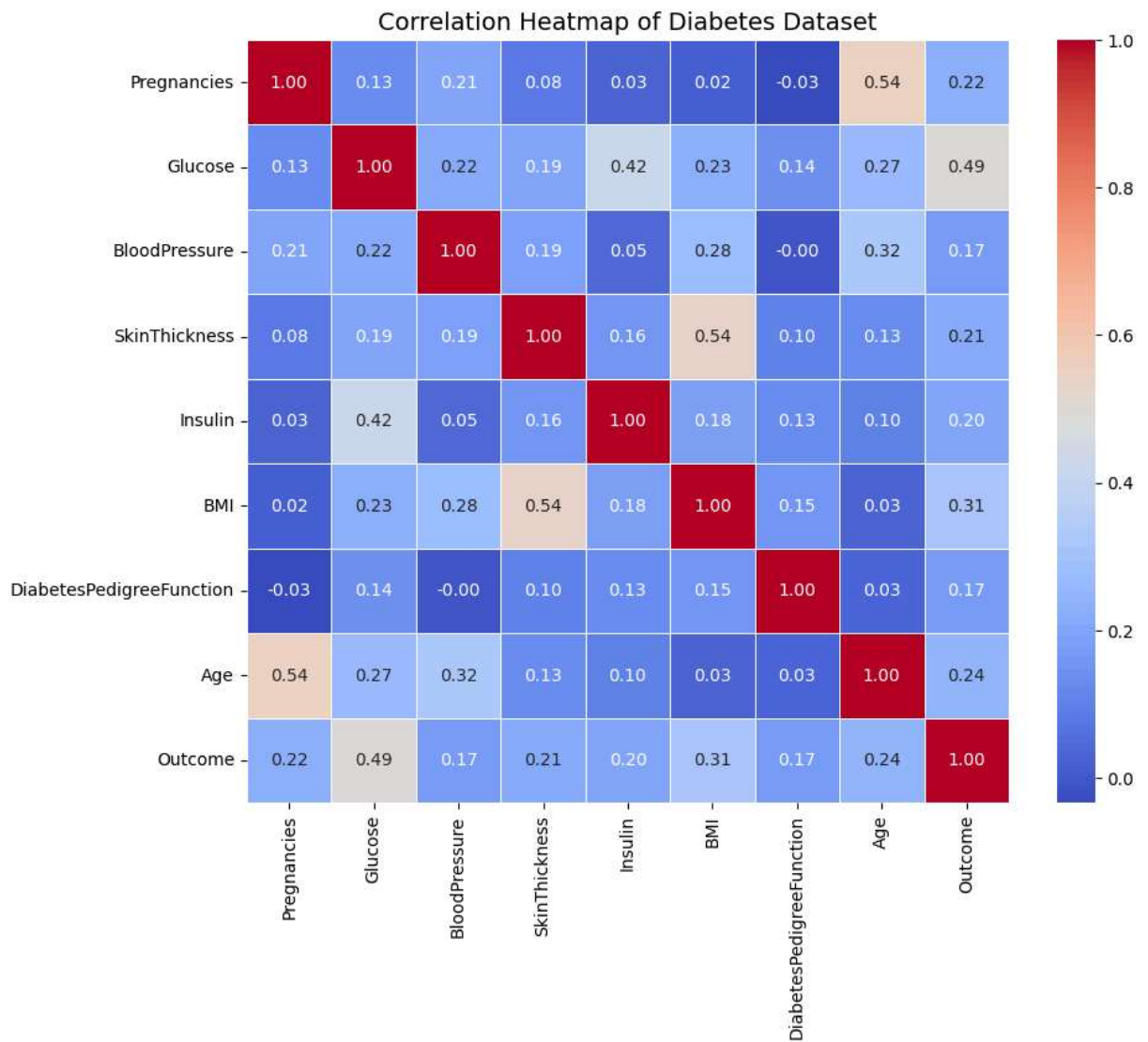
plt.show()
```



```
In [10]: plt.figure(figsize=(10, 8))
sns.heatmap(
    df.corr(),
    annot=True,           # show correlation values
    fmt=".2f",            # 2 decimal points
    cmap="coolwarm",      # color scheme
    linewidths=0.5,
    cbar=True,
    square=True
)

plt.title("Correlation Heatmap of Diabetes Dataset", fontsize=14)

plt.show()
```



In []:

Divide the data into input and output

```
In [11]: x = df.iloc[:, :-1]
x
```

Out[11]:

	Outcome	Age	DiabetesPedigreeFunction	BMI	Insulin	SkinThickness	BloodPressure
0	1	50	0.627	33.6	125.0	35.0	72.0
1	0	31	0.351	26.6	125.0	29.0	66.0
2	1	32	0.672	23.3	125.0	29.0	64.0
3	0	21	0.167	28.1	94.0	23.0	66.0
4	1	33	2.288	43.1	168.0	35.0	40.0
...	...	...	...	...	...	...	...
763	0	63	0.171	32.9	180.0	48.0	76.0
764	0	27	0.340	36.8	125.0	27.0	70.0
765	0	30	0.245	26.2	112.0	23.0	72.0
766	1	47	0.349	30.1	125.0	29.0	60.0
767	0	23	0.315	30.4	125.0	31.0	70.0

768 rows × 9 columns



In [12]: `y = df['Outcome']`
`y`

Out[12]:

```

0      1
1      0
2      1
3      0
4      1
..
763    0
764    0
765    0
766    1
767    0
Name: Outcome, Length: 768, dtype: int64

```

import KNN Model

In [13]: `from sklearn.neighbors import KNeighborsClassifier`
`from sklearn.preprocessing import StandardScaler`

In []:

Convert all features into Z score

```
In [14]: Scalar = StandardScaler()
```

```
In [15]: x_StandardScaler = Scalar.fit_transform(x)
```

```
In [16]: x_StandardScaler
```

```
Out[16]: array([[ 1.36589591,  1.4259954 ,  0.46849198, ..., -0.03198993,
                  0.86604475,  0.63994726],
                [-0.73212021, -0.19067191, -0.36506078, ..., -0.5283186 ,
                 -1.20506583, -0.84488505],
                [ 1.36589591, -0.10558415,  0.60439732, ..., -0.69376149,
                 2.01666174,  1.23388019],
                ...,
                [-0.73212021, -0.27575966, -0.68519336, ..., -0.03198993,
                 -0.02157407,  0.3429808 ],
                [ 1.36589591,  1.17073215, -0.37110101, ..., -1.02464727,
                 0.14279979, -0.84488505],
                [-0.73212021, -0.87137393, -0.47378505, ..., -0.19743282,
                 -0.94206766, -0.84488505]], shape=(768, 9))
```

Splitting the dataset into the Training set and Test set

```
In [17]: from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test = train_test_split(x_StandardScaler,y,test_size=0.3,r
```

Create KNN Model

```
In [18]: model = KNeighborsClassifier(n_neighbors=5,)
```

Fitting KNN on dataset

```
In [19]: model.fit(x_train,y_train)
```

```
Out[19]: KNeighborsClassifier
```

Parameters

```
In [20]: model.score(x_test,y_test)
```

```
Out[20]: 0.9913419913419913
```

Display classification_report

```

In [21]: from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay, classification_

In [22]: y_predict = model.predict(x_test)

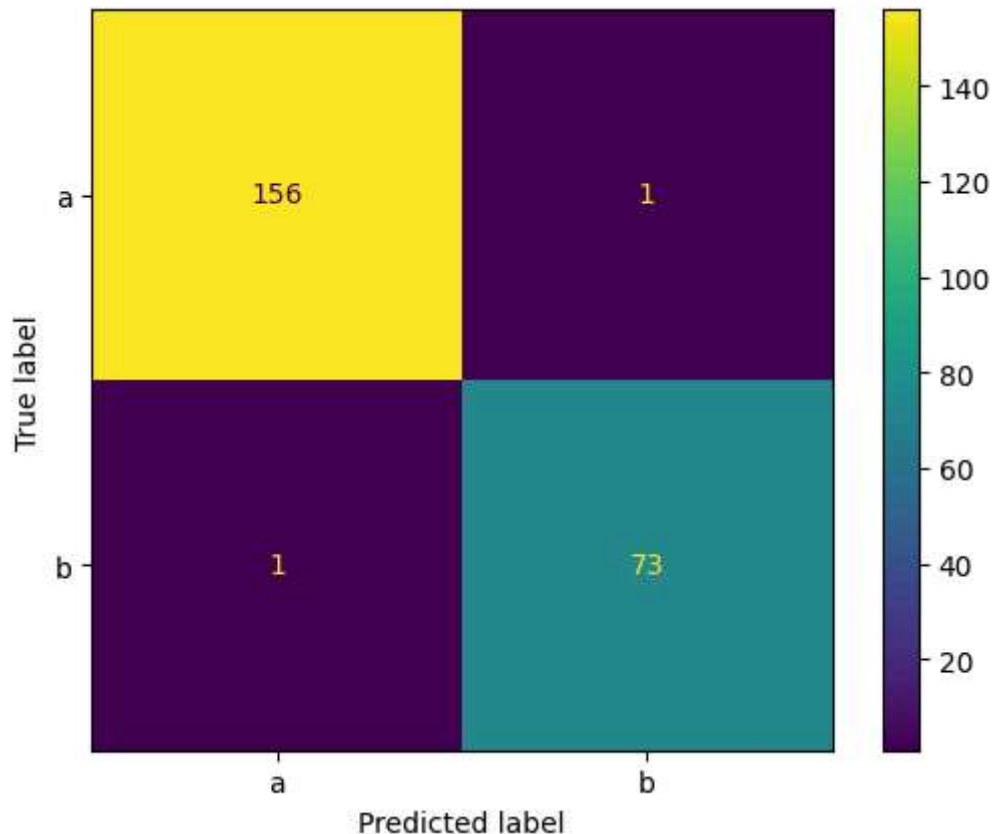
In [23]: cm = confusion_matrix(y_test, y_predict)

In [24]: display = ConfusionMatrixDisplay(cm, display_labels=['a', 'b'])

In [25]: display.plot()

Out[25]: <sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x1fc7fe9de80>

```



```

In [26]: print(classification_report(y_test, y_predict))

```

	precision	recall	f1-score	support
0	0.99	0.99	0.99	157
1	0.99	0.99	0.99	74
accuracy			0.99	231
macro avg	0.99	0.99	0.99	231
weighted avg	0.99	0.99	0.99	231

Use Cross Validation to find best hyperparameter for KNN

```
In [27]: from sklearn.model_selection import cross_val_score
```

```
In [28]: from sklearn.model_selection import KFold
```

```
In [29]: k = KFold(n_splits=10)
```

```
In [30]: for train_index, test_index in k.split(x_StandardScaler):
          x_train, x_test = x_StandardScaler[train_index], x_StandardScaler[test_index]
          y_train, y_test = y[train_index], y[test_index]
          model1 = KNeighborsClassifier()
          model1.fit(x_train, y_train)
          print(model1.score(x_test, y_test))
```

```
1.0
0.987012987012987
0.987012987012987
1.0
1.0
0.987012987012987
1.0
1.0
1.0
0.9736842105263158
```

```
In [41]: cross_val_score(model, x_StandardScaler, y, cv=k)
```

```
Out[41]: array([1.          , 0.98701299, 0.98701299, 1.          , 1.          ,
                0.98701299, 1.          , 1.          , 1.          , 0.97368421])
```

```
In [31]: import numpy as np
```

```
In [32]: x1 = np.arange(1,11).reshape(-1,1)
```

```
In [33]: y1 = np.array([0,1,0,1,0,1,0,1,0,1])
```

```
In [34]: x1
```

```
Out[34]: array([[ 1],
                [ 2],
                [ 3],
                [ 4],
                [ 5],
                [ 6],
                [ 7],
                [ 8],
                [ 9],
                [10]])
```

```
In [35]: y1
```

```
Out[35]: array([0, 1, 0, 1, 0, 1, 0, 1, 0, 1])
```

```
In [36]: for i,j in k.split(x1):
          print("<----->")
```

```
print(x1[i])  
print(x1[j])  
x_train,x_test = x1[i],x1[j]  
y_train,y_test = y1[i],y1[j]
```

```
<----->
[[ 2]
 [ 3]
 [ 4]
 [ 5]
 [ 6]
 [ 7]
 [ 8]
 [ 9]
 [10]]
[[1]]
<----->
[[ 1]
 [ 3]
 [ 4]
 [ 5]
 [ 6]
 [ 7]
 [ 8]
 [ 9]
 [10]]
[[2]]
<----->
[[ 1]
 [ 2]
 [ 4]
 [ 5]
 [ 6]
 [ 7]
 [ 8]
 [ 9]
 [10]]
[[3]]
<----->
[[ 1]
 [ 2]
 [ 3]
 [ 5]
 [ 6]
 [ 7]
 [ 8]
 [ 9]
 [10]]
[[4]]
<----->
[[ 1]
 [ 2]
 [ 3]
 [ 4]
 [ 6]
 [ 7]
 [ 8]
 [ 9]
 [10]]
[[5]]
<----->
```

```
[[ 1]
 [ 2]
 [ 3]
 [ 4]
 [ 5]
 [ 7]
 [ 8]
 [ 9]
 [10]]
[[6]]
<----->
[[ 1]
 [ 2]
 [ 3]
 [ 4]
 [ 5]
 [ 6]
 [ 8]
 [ 9]
 [10]]
[[7]]
<----->
[[ 1]
 [ 2]
 [ 3]
 [ 4]
 [ 5]
 [ 6]
 [ 7]
 [ 9]
 [10]]
[[8]]
<----->
[[ 1]
 [ 2]
 [ 3]
 [ 4]
 [ 5]
 [ 6]
 [ 7]
 [ 8]
 [10]]
[[9]]
<----->
[[1]
 [2]
 [3]
 [4]
 [5]
 [6]
 [7]
 [8]
 [9]]
[[10]]
```

```
In [37]: parameters = {'n_neighbors':(1,2,3,4,5,6,7,8,9,10,11), 'weights':('uniform', 'distan
```

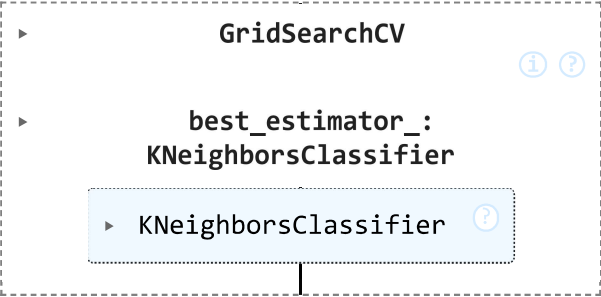
```
In [38]: from sklearn.model_selection import GridSearchCV
```

```
In [42]: m1 = KNeighborsClassifier()
```

```
In [44]: gs = GridSearchCV(m1,parameters)
```

```
In [45]: gs.fit(x_StandardScaler,y)
```

```
Out[45]:
```



```
In [46]: gs.best_params_
```

```
Out[46]: {'n_neighbors': 1, 'weights': 'uniform'}
```

```
In [ ]:
```

```
In [ ]:
```

```
In [ ]:
```

```
In [ ]:
```

```
In [39]: # Randomstate = 1
# Test size 0.3
#
```

```
In [ ]:
```